

WEB3 . DLT . SMART-CONTRACT ENGINEERING . ROLE-BASED CRYPTOGRAPHY

UniVerifyV2.0

A Decentralized Academic Certificate Verification Protocol Using Alphanume
Keyspace Mapping

B.Tech (Computer Science & Engineering) | Final-Year Capstone Project

An immutable, globally accessible, zero-latency ledger that supersedes centralized RDBMS verification pipelines

Solidity v0.8.0

Ethers.js v5.7.2

MetaMask Web3 Provider

Ethereum Sepolia Testnet

PHASE 1 - PRODUCTION LEDGER PROTOTYPE (Manual Administrative Curation via Remix IDE)

Problem Statement & Motivation

Conventional academic record tracking relies entirely on centralized relational database management systems (RDBMS) and legacy trails. Such architectures concentrate trust in a single authority, introducing systemic fragility across security, latency, and cost.

SINGLE POINT OF FAILURE

Centralized servers are exposed to malicious insider modification, external breaches, and downtime, creating a structural SPOF risk.

HIGH VERIFICATION LATENCY

Manual lookups incur a 7-14 day turnaround, throttling admissions, recruitment, and onboarding workflows.

RECURRING ADMINISTRATIVE COST

Redundant manual institutional audits impose substantial, continuous operational overhead.

TRIVIAL CREDENTIAL FORGERY

Unanchored PDFs and physical assets are visually falsifiable; forgery detection is near-impossible without cross-checks.

RESEARCH OBJECTIVE

Engineer a trust-minimized verification protocol that anchors certificate records to a public EVM ledger - rendering them immutable, queryable, and verifiable in constant time without a centralized intermediary.

Proposed Solution & System Specification

ARCHITECTURAL THESIS

UniVerify V2.0 anchors academic credentials directly onto the public Ethereum Sepolia Testnet. Once a record is mined, its state becomes cryptographically immutable and globally readable.

- **Immutability** Mined state cannot be retroactively altered or deleted.
- **Global Availability** Permissionless read access, 24x7, with no host server.
- **Zero-Latency Reads** Verification resolves in seconds via a gasless view call.
- **Tamper-Evidence** On-chain anchoring defeats document-level forgery.

PROJECT

UniVerify V2.0

DOMAIN

DLT . Web3 . RBAC

CONTRACT

Solidity v0.8.0

NETWORK

**Sepolia Testnet
(EVM)**

WEB3 LIB

Ethers.js v5.7.2

SIGNER

MetaMask Provider

NET EFFECT A 7-14 day, forgery-prone, centralized pipeline collapses into an instant, tamper-evident, decentralized O(1) lookup.

Two-Tier Phased Implementation Framework

Core verification logic is decoupled from automated transactional ingestion.

PHASE 1

CURRENT - 3RD YEAR

Production Ledger Prototype

Operates on a Manual Administrative Curation Paradigm to store and validate state mutability, permission guardrails, and read-latency.

- ✓ Registry Admin manually inputs records via the Remix IDE transaction dashboard.
- ✓ Fields entered by hand: Alphanumeric Reg. No., Full Name, Contact Name.
- ✓ The hash field is a MANUALLY-TYPED alphanumeric MOCK state-data-fingerprint placeholder only. No file is uploaded or hashed.
- ✓ Objective: cryptographically validate contract logic, not production hashing.

PHASE 2

ROADMAP - FINAL YEAR

Autonomous AI Agent Orchestration

A server-side Python / Node execution layer runs an autonomous agent that removes the human-in-the-loop bottleneck entirely.

- Ingests unstructured raw student logs and runs NLP normalization pipelines.
- Natively computes a deterministic SHA-256 / IPFS file-blob content hash.
- Signs & broadcasts state-changing transactions via Web3.py / Ethers.js using its own secure wallet.
- Real cryptographic certificate hashing is introduced HERE - not in Phase 1.

Algorithmic Inventions & Complexity Metrics

{ }

Alphanumeric Keyspace Mapping

State is bound via mapping(string => Certificate) to the unique Registration Number string (e.g. "2021CSE101") rather than sequential uint256 indices.

 $O(1)$

Constant-Time Retrieval

The keyspace structure bypasses iteration loops entirely, collapsing lookups from $O(N)$ linear scans to flat $O(1)$ constant time at any record volume.

RBAC

EVM-Layer Access Control

The constructor freezes the deployer address as an immutable admin. Unauthorized writes are reverted by require() before gas is committed to an invalid state.

GAS

Write/Read Cost Asymmetry

State mutations (issueDegree) incur gas; verification calls read an existing storage slot and therefore cost no gas.

COMPLEXITY

Array indexing: $O(N)$ linear scan, gas grows with N Keyspace mapping: $O(1)$ constant, flat cost

Smart Contract Source Engine

AcademicCertificates.sol

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract AcademicCertificates {
5     struct Certificate {
6         string studentName;
7         string courseName;
8         string ipfsHash; // mock placeholder (P1)
9     }
10
11     // RegNo string key -> O(1) lookup
12     mapping(string => Certificate) public certificates;
13     uint256 public totalIssued;
14     address public university;
15
16     constructor {
17         university = msg.sender;
18     }
19
20     function issueDegree(
21         string memory _regNo, _name,
22         string memory _course, _hash
23     ) public {
24         require(msg.sender == university,
25             "Only the University...");
26         certificates[_regNo] =
27             Certificate(_name, _course, _hash);
28         totalIssued++;
29     }
30 }
31

```

struct Certificate

Packs studentName, courseName & a hash-string field into one EVM storage record.

mapping(string=>...)

Reg. No. string is the lookup key -> O(1) reads/iteration.

university=msg.sender

Deployer frozen as the permanent, immutable admin owner.

require(...==university)

Reverts every unauthorized write at the EVM layer before gas commit.

ipfsHash (Phase 1)

Currently stores a MANUAL mock string; real SHA-256 / IPFS hashing is Phase 2.

Decentralized Frontend & Read-Path Discrimination

SERVERLESS STATE BRIDGE

The interface integrates an injected Web3 engine, binding to MetaMask through ethers.providers.Web3Provider(window.ethereum). No backend server mediates the verification request.

- **Web3Provider(window.ethereum)**
Injected MetaMask provider bridge.
- **contract.certificates(regNo)**
Isolated view call - reads an active storage slot, costs ZERO gas.
- **ABI + struct destructuring**
Decodes returned EVM struct fields into the UI model.
- **Uninitialized-Key Safety Filter**
if (!data.studentName || data.studentName.trim() === "")

DYNAMIC #result STATES

#result .success

✓ Certificate Verified

Resolved struct fields (studentName, courseName, hash) render into the verified panel.

#result .error

✗ Record Not Found

An unknown / mistyped key returns zero-state blanks; the filter intercepts the empty shape and renders this card.

EVM returns default "" for uninitialized mapping keys (no rever

End-to-End System Dataflow

01

Compilation

Solidity is compiled to EVM bytecode and an ABI JSON interface.

02

Deployment

Admin signs a contract-creation tx (blank "field") and pays gas to Sepolia.

03

Issuance (Write)

Admin enters fields in Remix; MetaMask signs; record is mined. Costs gas.

04

Verification (Read)

Verifier submits a Reg. No.; a gasless view call returns the record instantly.

- **WRITE PATH** state mutation, requires admin signature & gas (SepoliaETH).
- **READ PATH** gasless view call, permissionless, resolves in $O(1)$ constant time.

The Hash Field: Current Reality vs. Final-Year Goal

Disambiguating the ipfsHash struct field across the project timeline.

PHASE 1 - NOW (3RD YEAR)

Manual Mock Placeholder

- The ipfsHash field is populated by HAND in Remix with alphanumeric string.
- No certificate file is uploaded; no IPFS pinning occurs.
- No SHA-256 / cryptographic digest is computed anywhere in the pipeline.
- Purpose: a data-fingerprint PLACEHOLDER to exercise storage layout and read path.

PHASE 2 - FINAL YEAR

Real Cryptographic Hashing

- An autonomous agent computes a deterministic SHA-256 / IPFS file-blob hash from the actual certificate.
- The content hash is generated off-chain, then anchored on-chain programmatically.
- Web3.py / Ethers.js sign & broadcast the write from the agent's wallet.
- This upgrades the placeholder into a true tamper-evident data fingerprint.

VIVA NOTE: The field is named ipfsHash for forward-compatibility, but in the current prototype it carries a manually-entered number.

Technology Stack & Engineering Outcomes

TECHNOLOGY STACK

Smart Contract

Solidity v0.8.0

Execution Network

Ethereum Sepolia Testnet (EVM)

Web3 Library

Ethers.js v5.7.2

Wallet / Signer

MetaMask Injected Provider

Data Fingerprint (P1)

Manual mock string in Remix IDE

Hashing Roadmap (P2)

SHA-256 / IPFS via autonomous agent

ENGINEERING OUTCOMES

- ✓ Immutable, tamper-evident on-chain records
- ✓ $O(1)$ constant-time verification at scale
- ✓ Zero-latency, 24x7 permissionless global reads
- ✓ No single point of failure (no central server)
- ✓ Gasless verification path for public verifiers
- ✓ EVM-enforced admin-only write authorization



CONCLUSION

Trust, anchored on-chain.

UniVerify V2.0 reframes certificate verification from a multi-day, centralized, forgery-prone process to a decentralized, constant-time lookup. The Phase-1 prototype proves the contract's state mutability, gasless read path under a manual curation paradigm.

Next Milestone (Final Year)

Phase 2 introduces the Autonomous AI Agent: NLP-normalized ingestion, deterministic SHA-256 / IPFS content addressing, transaction signing - eliminating the human-in-the-loop entirely.